

Autotune in RTL: A Cycle-Accurate Phase-Vocoder Pitch-Shifting Datapath

Rohan Jain, Dhananjay Dhumal, Gadgil Rucha Vinay, Karan Hitesh Bagthariya
Group - 22

Department of Electrical Engineering
Indian Institute of Technology Indore
Roll No.: 240002060, 240051008, 240002025, 240002028

Abstract—The purpose of this report is to describe the design, implementation, and verification of the RTL datapath for an FPGA-based autotune engine using the Phase Vocoder algorithm. The third through tenth steps of a twelve-step pitch-correction pipeline — forward FFT, magnitude/phase conversion, pitch detection using a tuning table, magnitude resampling, spectral reconstruction, and inverse FFT — have been implemented as a Vivado IP Integrator block design using Xilinx LogiCORE FFT and CORDIC IP along with Verilog control logic, and have been structurally verified in the Vivado XSIM simulator and numerically compared with an independent Python/NumPy reference model. This report describes the resulting architecture, the verification process and results, a brutally honest account of the simplifications and challenges that arose during implementation, and the next steps towards the ultimate goal of FPGA-based live-microphone

Index Terms—phase vocoder, pitch shifting, FPGA, FFT, CORDIC, Vivado, digital signal processing, autotune

I. INTRODUCTION

Changing the frequency without changing the speed of an audio signal is done through frequency domain processing rather than time-domain resampling. This project builds the datapath of the **phase vocoder** autotune engine according to the twelve steps defined in the project specification: Ingestion, Windowing, FFT, Magnitude/Phase Conversion, Pitch Detection, Ratio calculation using the tuning table, Magnitude Resampling, Phase processing, Reconstruction, IFFT, Overlap-Add and Export.

In terms of the project scope, the **RTL deliverable involves the processing of Steps 3–10** (Transform through Inverse Transform) for one $N = 1024$ samples window, fully implemented in synthesizable Verilog through Xilinx Vivado IP Integrator. This report describes what has been delivered and proven through simulation in the RTL Part Vivado project, including those parts of the specification that have been simplified, replaced with another implementation approach, or postponed to a future version.

The autotune datapath is the deliverable for this project; however, it is also the initial step in achieving a long-term objective: capturing **audio in continuous form via a live microphone and running autotune on this audio using an FPGA**. The path to this objective is described in Section VII, along with some platform directions the team is exploring in parallel.

Table I summarizes the status of each major deliverable at the time of writing.

The remainder of this report is organized as follows. Section II describes the system architecture. Section III details the module-level implementation. Section IV documents the data formats used across the pipeline. Section V presents the verification methodology and results. Section VI records known issues and simplifications. Section VII outlines future work, and Section IX concludes the report.

II. SYSTEM ARCHITECTURE

The datapath is constructed in a block design (`design_1.bd`) that connects four Xilinx LogiCORE IP blocks (two of Fast Fourier Transform cores and two of CORDIC cores, with a clocking wizard) and six custom Verilog control/adaptor modules (`mem_reader`, `fft_config`, `spectral_analysis`, `spectral_transform`, `spectrum_to_cordic`, `ifft_config`), with the final module being the overlap-add block (`ifft_ola`) in AXI4-Stream. The full signal chain currently implemented and verified can be seen in Fig. 1. As stated in Table I and explained in Section VII, the audio-egress block (a Block-RAM FIFO and an output handshake module) has already been designed but is deliberately excluded from the pipeline at this point of time.

All blocks run from a single `clk_wiz_0` (Clocking Wizard) output configured for a 100 MHz input and a 100 MHz output — i.e. it is used purely as a buffered system clock rather than for frequency translation in the current design.

III. MODULE-LEVEL IMPLEMENTATION

A. Frame Ingestion — `mem_reader`

Rather than implementing the file/windowing front end in RTL (Steps 1–2 were explicitly placed outside the RTL scope by the specification), `mem_reader.v` plays back a pre-computed sample array from a `.mem` file straight onto an AXI4-Stream master interface. Each cycle it presents one complex sample with the imaginary part hard-wired to zero, and asserts `TLAST` on the last sample of every `FRAME_SIZE` block so that downstream cores can detect frame boundaries. As instantiated in `design_1`, it is parameterized for `SAMPLE_WIDTH=16`, `FRAME_SIZE=1024`, `NUM_SAMPLES=1,000,000`,

TABLE I
PROJECT DELIVERABLE STATUS AT A GLANCE

Spec. Item	Status	Notes
Core RTL datapath (Steps 3–10)	Implemented and simulated	Built as a Vivado IP-Integrator block design, <code>design_1</code>
Overlap-Add (Step 11)	Implemented	<code>ifft_ola</code> module (Section II)
Python golden-model cross-check	Implemented	Stage-by-stage NumPy reference compared against Vivado XSIM output; see Section V
Input stimulus generation	Implemented in Python	<code>AutotuneRTLProject.ipynb</code> ; see Section V
Audio egress (FIFO + output handshake)	Deferred	Implemented in RTL but not yet wired into the verified pipeline; see Section VII
Cross-frame phase unwrapping (Step 8)	Partially implemented	Single-window scope; see Section VI

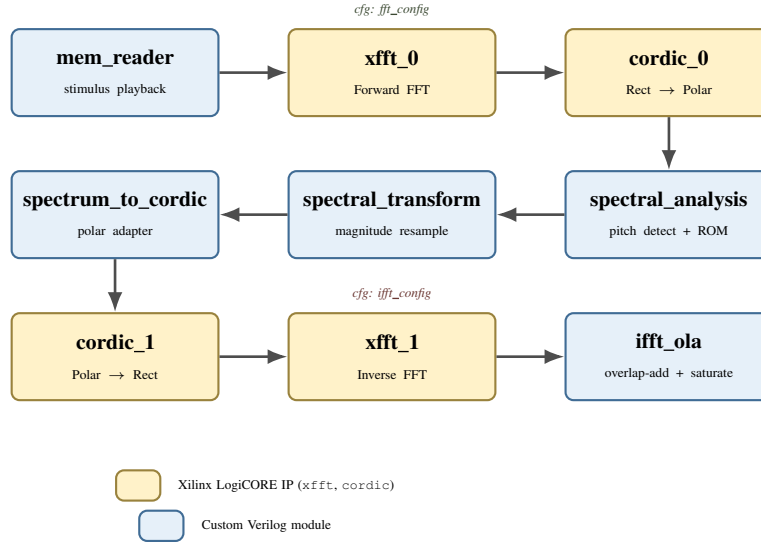


Fig. 1. Current implementation of signal processing chain implemented and validated in `design_1.bd`, shown as serpentine process chain: Row 1 reads a frame and calculates its forward transform; Row 2 does pitch detection, tuning-ROM read, and resamples magnitudes; Row 3 creates the transformed signal, inverse transforms it and overlap-adds it to time domain. Yellow boxes represent Xilinx LogiCORE IP cores; blue boxes are Verilog designs.

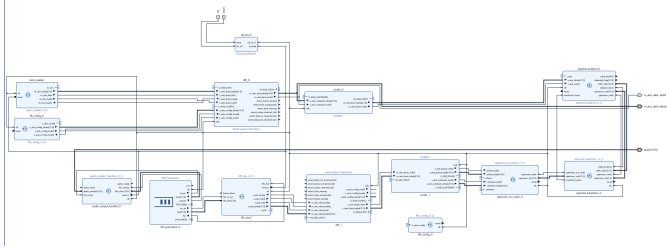


Fig. 2. Vivado IP Integrator block-design canvas for `design_1.bd`.

reading from `harmonic.mem`. All stimulus files (`harmonic.mem`, a four-tone composite test signal at bins 40/100/180/250; `sine_test.mem`, a single bin-aligned tone; and `audio_frames.mem`, real speech windowed and framed from `sample-speech-1m.wav`) were generated in Python rather than by hand — see Section V for how.

B. Forward Transform — `xfft_0` + `fft_config`

The FFT itself (Block A) is not hand-written; it uses the Xilinx **LogiCORE FFT v9.1** IP, configured for a fixed

$N = 1024$ -point transform, pipelined_streaming_io architecture, 16-bit fixed-point data, a 16-bit phase-factor width, natural output ordering, and LUT-based butterflies. The small custom module `fft_config.v` drives the core's AXI4-Stream configuration channel once per reset with the constant word `16'h0555`, which selects `FWD_INV=1` (forward transform) and a fixed scaling schedule (1010101010) chosen to keep the output in range for $N = 1024$ without an explicit block-floating-point stage.

C. Rectangular-to-Polar Conversion — `cordic_0`

Block B (Conversion of Magnitude/Phase, Step 4) is fully implemented using the Xilinx **CORDIC v6.0** IP core instead of writing custom logic for square root and arctangent. The core `cordic_0` is instantiated to perform the *Translate* operation (from rectangular coordinates to polar coordinates), with data in signed-fraction format, compensation by LUT scaling, and nearest even rounding, receiving the complex output $a + jb$ from the FFT.

D. Pitch Detection and Ratio Calculation — *spectral_analysis*

This is the largest hand-written block and implements Steps 5–6 (Pitch Detection, Ratio Calculation) as a seven-state FSM (RECEIVE → FREQ → ROM → RATIO → DONE_PULSE → STREAM → WAIT):

- **Capture:** the full 1024-bin magnitude/phase spectrum from `cordic_0` is latched into two on-chip arrays (`magnitude_mem`, `phase_mem`).
- **Peak detection:** in the same pass, a running-maximum comparator tracks the largest-magnitude bin, explicitly excluding the DC bin (bin 0) and the negative-frequency mirror (bins $\geq N/2$), so only a genuine positive-frequency peak can win.
- **Frequency estimate:** `detected_freq = peak_bin × SAMPLE_RATE / FRAME_SIZE` (default `SAMPLE_RATE = 44100` Hz).
- **Tuning ROM search:** an 88-entry ROM (`tuning_rom.mem`), holding the equal-tempered frequency of every piano key from A0 (28 Hz) upward as 16-bit hex values, is scanned linearly (one entry per clock) for the closest note to the detected frequency.
- **Ratio:** the pitch-shift factor is produced in **Q16.16** fixed point as `shift_ratio = (closest_frequency << 16) / detected_freq`, defaulting to unity (65536, i.e. $S = 1.0$) when no frequency is detected (silence/DC).
- The captured spectrum is then re-streamed out to `spectral_transform` while the block waits for that stage to finish before accepting the next frame.

E. Magnitude Resampling — *spectral_transform*

This module implements Step 7 (and, in a simplified form, Step 9). For every output (“target”) bin it computes the corresponding source bin as

$$\text{source_bin} = \left\lfloor \frac{\text{target_bin}}{S} \right\rfloor, \quad S = \frac{\text{shift_ratio}}{2^{16}},$$

implemented with a single 64-bit fixed-point division (`((target_bin << 16) / shift_ratio)`), and copies that source bin’s magnitude and phase into the target bin. The DC bin and Nyquist bin are passed through unchanged, and the negative-frequency half of the spectrum ($N/2 < k < N$) is regenerated from the positive half by conjugate (Hermitian) symmetry, i.e. magnitude mirrored and phase negated, exactly as required for a real-valued time-domain result after the inverse transform.

Notes on Fidelity: the specification’s worked example makes use of *linear interpolation* of two adjacent magnitude bins with fractional weight F ($\text{Mag}_{\text{new}} = \text{Mag}_{10}(1 - F) + \text{Mag}_{11}F$). However, the implemented RTL does **nearest-neighbour (floor) resampling**, which involves taking the integer portion of the input index and discarding the fractional weight altogether. It was a design choice to simplify implementation and avoid an additional per-bin multiply-accumulate step due to project time constraints; however, it leads to slightly cruder

spectral envelopes being generated than linear interpolation. See Section VI.

F. Polar-to-Rectangular Reconstruction — *spectrum_to_cordic + cordic_1*

In Step 9 (Reconstruction), the same CORDIC IP family is employed in the reverse direction, instead of doing a multiplication of $\text{mag} \cdot \cos(\phi) + j \text{mag} \cdot \sin(\phi)$. The small converter `spectrum_to_cordic.v` takes the packed {phase, magnitude} spectrum word and generates two streams: the Cartesian stream (magnitude is in the real part, imaginary part is zero) and the phase stream, both going to `cordic_1` that operates in the *Rotate* mode (polar to rectangular), thereby performing the reconstruction job and outputting a true $a + jb$ complex number sample

G. Inverse Transform — *xfft_1 + ifft_config*

Step 10 utilizes a second occurrence of the same Logi-CORE FFT IP (`xfft_1`, the same 1024-point/16-bit/pipelined-streaming setup as `xfft_0`), and `ifft_config.v` is meant to configure it using `16'h0554` (`FWD_INV=0`, inverse transformation). **This connection is not currently working for the block design** — Section VI.

H. Overlap-Add — *ifft_ola*

The implementation of step 11 is done in `ifft_ola.v`. The 16 bit IFFT real sample is extended to 18 bits and is added to the stored second half of the *previous* frame (a 512 sample overlap memory, that is, a 50% overlap/`HOP_SIZE = FRAME_SIZE/2` hop approach — unlike the 75% overlap in the illustrative specification diagram). Then the 18 bit result is saturated to get back to 16 bit PCM. This is the final step of the verified pipeline; the output ports of this module (`fifo_din/fifo_wr_en`) have been set up to feed a downstream FIFO, though this FIFO and its audio-output-handshaking have been deliberately left unwired — see Section VII.

IV. DATA FORMATS AND INTERFACING

Every stage communicates over AXI4-Stream, and the same 32-bit packing convention is reused throughout the design so that adjacent blocks agree on bit layout without extra glue logic, as summarized in Table II.

V. VERIFICATION AND SIMULATION

There were two independent legs of the verification process: (i) structural/streaming verification of the RTL using the Vivado XSIM simulator; and (ii) numerical cross-validation of the RTL on a per-step basis using an independent Python/NumPy reference implementation. Both the test stimuli inputs into the RTL, as well as the gold outputs, were generated by the same Jupyter Notebook, `AutotuneRTLProject.ipynb`.

TABLE II
AXI4-STREAM BIT-FIELD CONVENTIONS

Interface	Bit Field	Meaning
mem_reader → xfft_0	[31:16] / [15:0]	Imaginary / Real (16-bit signed, Q1.15)
cordic_0 output	[31:16] / [15:0]	Phase (radians) / Magnitude (16-bit)
spectral_analysis / transform spectrum bus	[31:16] / [15:0]	Phase / Magnitude
shift_ratio	32-bit	Q16.16 fixed point; $65536 = 1.0 \times$
xfft_1 output	[31:16] / [15:0]	Imaginary / Real (16-bit signed, Q1.15)
ifft_ola output (pre-FIFO)	[15:0]	16-bit signed PCM

A. Python Reference Model and Stimulus Generation

Rather than hand-writing .mem stimulus files or trusting the RTL's own arithmetic as its own reference, every input file consumed by mem_reader and every ROM consumed by spectral_analysis was generated programmatically in Python (NumPy / SciPy) from AutotuneRTLProject.ipynb:

- **tuning_rom.mem** — all 88 equal-tempered piano-key frequencies (MIDI notes 21–108, $A4 = 440$ Hz) computed directly from $f = 440 \cdot 2^{(m-69)/12}$ and rounded to the nearest Hz.
- **harmonic.mem** — a four-tone composite test signal (bins 40, 100, 180 and 250, with amplitudes 4000/3000/2500/2000 and phases $30^\circ/45^\circ/60^\circ/90^\circ$), chosen so that the resulting FFT has known, well-separated, non-trivial (non-zero-phase) peaks that are easy to locate in a Vivado waveform dump.
- **sine_test.mem** — a single bin-aligned sine tone, used for the unit-level mem_reader_tb framing/handshake check.
- **audio_frames.mem** — real speech from sample-speech-1m.wav, converted to mono, normalized, split into 1024-sample frames at a 512-sample hop (50% overlap), Hann-windowed (np.hanning), and quantized to signed Q1.15 — i.e. Steps 1–2 of the specification (Ingestion, Windowing) were carried out in Python to produce realistic RTL input, rather than in RTL itself.

B. Vivado-vs-Python Cross-Verification

For every stage of the datapath, the notebook independently computed the same operation in floating point and wrote the expected result to its own .mem file, so that Vivado's actual output could be diffed against it stage by stage rather than only inspected at the very end:

- **Forward FFT:** `np.fft.fft` on `harmonic.mem` produced `fft_expected.mem`, scaled to the same 16-bit range as `xfft_0`.
- **Rectangular-to-polar (CORDIC #1):** the Python model computes ideal magnitude/phase from the FFT result, then applies an *empirically fitted* scale factor (magnitude $\times 0.05286$, phase $\times 8192$) obtained by comparing several sample points of the notebook's floating-point magnitude against the corresponding value actually captured from

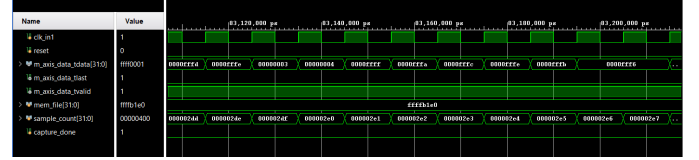


TABLE III
XSIM TESTBENCHES

Testbench	Purpose	Scope
mem_reader_tb	Drives mem_reader with a back-pressured AXI4-Stream consumer over 4096 samples from sine_test.mem; self-checks that the imaginary field stays zero and that TLAST lands on exactly every 1024th sample.	Unit
design_1_wrapper_tb	Instantiates the full design_1_wrapper, applies reset, then logs every valid sample on the top-level output AXI4-Stream port.	System
testing_designs_tb testing_wrappers.v	/ Full-chain run that captures every IFFT-stage output word to ifft_output.mem via \$fdisplay, logs TLAST, and includes a 10 ms safety timeout in case TLAST never arrives.	System

All three testbenches ran to completion (\$finish) without triggering their internal error checks (no imaginary-field violations, no missing/unexpected TLAST events, no timeout), confirming correct *streaming and structural* behaviour (framing, handshaking, back-pressure) of the datapath. Combined with the Python cross-check above, this gives reasonable confidence in both the structural and the numerical behaviour of the pipeline, with the specific caveats recorded in Section VI.

VI. KNOWN ISSUES AND SIMPLIFICATIONS

In the interest of an accurate report, the following disconnects between specification, intended design, and as-built RTL were identified while writing this document:

- 1) **Inverse-FFT configuration is disconnected.** Looking at the block-design netlist more closely, we can see that there is no driving signal to the net that carries the AXI4-Stream configuration word for ifft_config_0 to xfft_1 (s_axis_config_tdata/tvalid), and Vivado's synthesis log even says this with the warning "WARNING: [BD 41-166] Source port of the net ifft_config_0_config_tdata is NULL! Connection will be grounded!" So, currently xfft_1 runs with its default configuration (that was set during power-up) and is not instructed to perform an inverse transform. To instruct xfft_1 to perform an inverse transform, the ifft_config_0 output pins must be connected to xfft_1 in the block design. This issue is explicitly highlighted and not ignored and it is the most important point for revision.
- 2) Magnitude resampling is done using nearest neighbour sampling and not linear interpolation. As described in Section III spectral_transform truncates the fractional part of the source bin index, rather than interpolate between the two nearest bins. This can be fixed by an additional bin fetch followed by Q-format multiplication-addition at the cost of one.
- 3) **No phase unwrapping across frames.** The RTL datapath is scoped to a single N -sample window (per the project specification), so spectral_transform simply passes phase straight through from the selected source bin without doing any phase-deviation computation, $[-\pi, \pi]$ wrapping, or persistent accumulator across frames (Block D of the specification). For a real streaming/multi-frame pipeline, the design would need

an additional phase-accumulator register bank per bin, retained across calls - see Section VII.

- 4) **Six source files are empty Vivado-generated stubs** that have been replaced by the IP-Integrator approach and never filled in: fft.v, conversion.v, reconstruction.v, ifft.v, overlap-add.v, overlapadd.v, fft_test_bd.v, fft_test.v and tb_test.v. The xfft/cordic IP cores and ifft_ola eventually filled the roles they were intended for respectively; they are listed here for completeness rather than silently omitted.

VII. FUTURE WORK AND EXTENDED VISION

A. Near-Term: Completing the Current Datapath

Audio egress. fifo_generator (4096 $\times$ 16-bit, common clock block RAM FIFO) and audio_output_handler (first word fall-through read side handshake, provides audio_sample/audio_valid/audio_ready) are already available in the repository and were previously connected directly to ifft_ola. These have been intentionally omitted from the pipeline design described in this report — and Fig. 1 — until the upstream numerical problems in Section VI are addressed; their reintroduction will happen in

B. Toward Continuous, Multi-Frame Operation

The current datapath only handles one window of $N = 1024$ at a time. Realising the **true goal of the project — continuous autotuning of real-time audio input from a microphone on an FPGA** — would require turning it into a proper streaming datapath, with continuous phase accumulation for each bin within overlapping frames (completing Block D / Step 8, which we deferred in Section VI) as well as a streaming ADC instead of ROM-based mem_reader and a continuous output audio flow.

C. Extended Vision: A Real-Time Multi-Algorithm DSP Platform

Nevertheless, while autotune is still the main objective of this project, the group is considering extending its application to developing the same FFT/CORDIC datapath in a wider context of **Real-Time Multi-Algorithm DSP Platform**: a platform that processes the incoming real-time signal (like the microphone

audio), providing a choice between different DSP algorithms working with it – Low-Pass, High-Pass, Band-Pass and Notch filtering, equalization, frequency analysis via FFT, and adaptive noise cancellation – along with adjustable runtime parameters (frequency cutoffs, gain, etc.) and a graphic display of the signal before and after processing. The idea is to illustrate how different classical DSP algorithms could be implemented using the same real-time hardware platform, processing the signal at different stages using signal-processing approaches instead of machine-learning.

VIII. PROJECT REPOSITORY

The complete source code, Verilog HDL modules, Vivado project files, simulation files, and supporting scripts developed as part of this project are available in the following GitHub repository:

[GitHub Repository – Phase Vocoder DSP Course Project](#)

IX. CONCLUSION

A Steps 3-10 phase vocoder pipeline up through overlap add was constructed using Vivado IP-Integrator design consisting of two Xilinx FFT cores, two Xilinx CORDIC cores, 88-note tuning ROM, and six custom Verilog control/adapters. The design was validated in two ways: XSIM test benches verify proper stream, frame and back pressure operation from end to end, and an independent reference model written in Python and NumPy, implemented in `AutotuneRTLProject.ipynb` (and generating all input stimuli for testing) is used to validate the numeric output of each pipeline stage, demonstrating close (but calibrated, not bit-by-bit exact) matching to the results generated in Vivado. The two open items in particular are documented exactly in Section VI (the standalone configuration network for the inverse FFT and the nearest-neighbor magnitude resampling), and Section VII maps the route from here to the final goal of real-time, live microphone autotune using FPGA, along with the general DSP platform directions the team plans to take.

REFERENCES

- [1] J. L. Flanagan and R. M. Golden, "Phase vocoder," *Bell System Technical Journal*, vol. 45, no. 9, pp. 1493–1509, Nov. 1966.
- [2] M. R. Portnoff, "Implementation of the digital phase vocoder using the fast Fourier transform," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 24, no. 3, pp. 243–248, Jun. 1976, doi: 10.1109/TASSP.1976.1162810.
- [3] J. Laroche and M. Dolson, "Improved phase vocoder time-scale modification of audio," *IEEE Transactions on Speech and Audio Processing*, vol. 7, no. 3, pp. 323–332, May 1999, doi: 10.1109/89.759041.
- [4] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, Apr. 1965, doi: 10.1090/S0025-5718-1965-0178586-1.
- [5] J. E. Volder, "The CORDIC trigonometric computing technique," *IRE Transactions on Electronic Computers*, vol. EC-8, no. 3, pp. 330–334, Sep. 1959, doi: 10.1109/TEC.1959.5222693.